

SENSE-LINK — Technical Documentation

1. Abstract

SENSE-LINK is an offline, on-device communication aid that bridges signed and spoken language in both directions. A signer signs to the front or rear camera; the device tracks the hands, classifies gestures into semantic tokens, rewrites those tokens into a fluent sentence with an on-device language model, and speaks the sentence aloud. A hearing person's spoken reply is transcribed to text and accompanied by a haptic cue. The complete pipeline — vision, language and speech — executes locally; no audio, video or text leaves the device, and the application functions in airplane mode.

This document describes the system architecture, the end-to-end data flow, the on-device and offline design, the permission and security model, performance characteristics, and the engineering decisions behind them.

2. Problem statement

People who rely on sign language face a persistent communication gap with people who do not sign. Existing mobile tools address it poorly:

- **Grammar mismatch.** Sign languages are spatial and use topic-comment structure; they do not map word-for-word onto spoken language. Tools that translate sign tokens literally produce telegraphic, undignified output (for example "water give need" instead of "Could I have some water, please?").
- **Connectivity dependence.** Most translation and transcription tools offload computation to a server. They become unusable in exactly the environments where they are most needed: underground transit, basements, clinics, and dense crowds where bandwidth collapses.
- **Thermal and power cost.** Running continuous computer vision while keeping a cellular radio active heats the device and exhausts the battery quickly, which makes sustained use impractical.

The design brief was therefore: produce dignified, full-sentence output; run with zero network dependency; and keep the computational footprint low enough for sustained handheld use.

3. Goals and non-goals

Goals

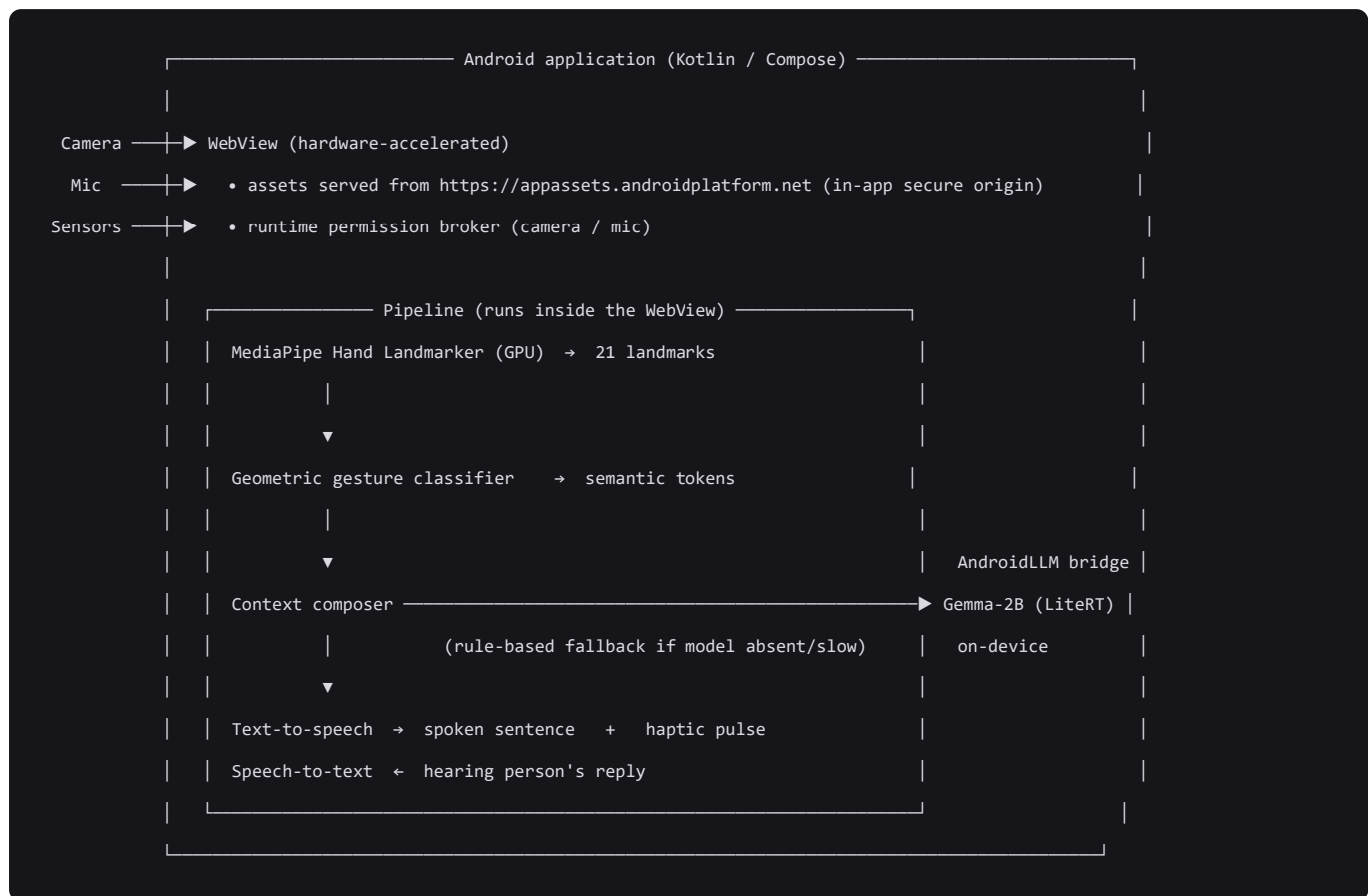
- Two-way communication: sign → speech, and speech → text.
- 100% on-device execution, including the language model.
- Graceful degradation: the app must never hard-fail during live use.
- A native, installable Android application.

Non-goals (this iteration)

- Full continuous sign-language translation across an unrestricted vocabulary. The current system recognises a curated gesture lexicon and composes from it.

- Cloud-assisted recognition or model hosting of any kind.

4. System overview



Every box runs locally. The only process boundary inside the device is the JavaScript ↔ Kotlin bridge between the in-WebView pipeline and the native language-model runtime.

5. Components

5.1 Application shell

The app is a single-activity Jetpack Compose application. The activity hosts a `WebView` through an `AndroidView` interop node. Two design choices make the embedded web pipeline behave like native code:

- **Secure in-app asset origin.** Rather than loading from `file://` (which blocks ES modules,

WebAssembly streaming and secure-context APIs such as `getUserMedia`), assets are served through a `WebViewAssetLoader` mapped to `https://appassets.androidplatform.net/assets/`. A custom path handler returns each asset with the correct `Content-Type` — notably `text/javascript` for `.mjs` and `application/wasm` for `.wasm`, which the default handler mislabels and which would otherwise silently break module loading and the vision runtime.

- **Robust media permissions.** When the page calls `getUserMedia`, the `WebView` raises a permission request. The shell only grants it after confirming the corresponding OS permission is actually held; if it is not, the shell requests it on demand and grants the web-layer request once the user approves. The activity is

resolved by unwrapping the Compose context chain, so the broker is always reachable regardless of how the host context is wrapped.

The activity keeps the screen awake during use, pauses and resumes the WebView with the lifecycle to release the camera when backgrounded, and locks to portrait.

5.2 Vision pipeline

Hand tracking uses MediaPipe Hand Landmarker in video mode on the GPU delegate, configured for a single hand. Each processed frame yields 21 three-dimensional landmarks. The detector is driven from a `requestAnimationFrame` loop; per-frame inference latency is measured with an exponential moving average and surfaced live in the interface. The landmark skeleton is drawn as an overlay so the user gets immediate feedback that tracking is active.

5.3 Gesture classifier

Classification is purely geometric and therefore deterministic and inexpensive. From the 21 landmarks the classifier computes:

- **Finger extension**, by comparing each fingertip's distance from the wrist against the distances of the intermediate (PIP) and base (MCP) joints.

- **Thumb extension**, from the lateral distance between the thumb tip and the index base.

- **Pinch / contact**, from the thumb-tip to index-tip distance, normalised by hand scale.

These features map to a curated lexicon of seven gestures (open palm → HELLO, fist → HELP, point up → WATER, victory → PLEASE, thumbs up → YES, OK sign → COFFEE, pinch → GO). To avoid jitter, a gesture must be held for a number of consecutive frames before it commits, and a cooldown prevents the same token from repeating on every frame. The hold threshold scales with a user-adjustable confidence setting.

5.4 Context / language layer

Committed tokens accumulate in a buffer. After a brief pause the buffer is composed into a sentence by one of two engines:

- **On-device LLM (primary)**. Gemma-2B (4-bit) runs through LiteRT via the MediaPipe LLM Inference runtime, behind a native bridge exposed to the page as `AndroidLLM`. The page sends the token list and target locale; the bridge runs inference on a background thread with a low-temperature, instruction-style prompt that asks for a single natural, polite sentence, and returns the result through a callback. Model weights live in the app's storage and are loaded once at startup.

- **Deterministic composer (fallback)**. A rule-based composer maps common token combinations to polished phrases and otherwise assembles a grammatical sentence from token glosses. It requires no model and runs instantly.

Selection is automatic: if the model is loaded the LLM path is used; otherwise the composer runs. A watchdog timer guarantees that a sentence is produced even if inference stalls, so the user-facing behaviour is bounded regardless of device speed. The interface labels which engine produced each sentence.

5.5 Speech output and input

Speech output uses the platform speech-synthesis engine. Voice selection follows the configured locale and an optional voice-profile preference, and playback rate is adjustable. Each spoken sentence is paired with

haptic pulses marking start and completion.

Speech input (the reverse, hearing-to-signer loop) uses the platform speech-recognition API to transcribe a reply to on-screen text with a haptic acknowledgement. (A native recognizer bridge with an offline language pack is the planned replacement for environments where the embedded recognizer is unavailable; see §11.)

5.6 Haptics and sensor fusion

Haptic feedback uses the device vibrator with distinct patterns for token capture, sentence start and sentence completion. A shake-to-clear gesture is implemented with a window-averaging accelerometer detector: the magnitude of recent acceleration samples is averaged over a sliding window, and a sharp shake above threshold (rate-limited by a cooldown) clears the active workspace.

6. End-to-end data flow

A representative interaction, "OK sign" followed by "victory":

1. The camera frame is delivered to MediaPipe, which returns 21 landmarks (single-digit to low tens of milliseconds on the GPU delegate).

1. The classifier reads the landmarks as an OK sign and, after the hold threshold, commits the token

COFFEE. A token-capture haptic fires.

1. A second gesture commits **PLEASE**.

2. After a short pause the buffer **[COFFEE, PLEASE]** is sent to the language layer.

3. The on-device model returns "Could I have a coffee, please?" (or the deterministic composer produces an equivalent sentence if no model is present).

1. The sentence is displayed, spoken aloud, paired with start/complete haptics, and logged to the on-device session history.

The reverse direction: the hearing person's reply is transcribed and shown as incoming text with a haptic cue. A sharp shake clears the workspace for the next exchange.

7. Offline and on-device design

The application carries every runtime dependency inside the APK so that it works with no network:

- The MediaPipe vision runtime (ES-module bundle plus the SIMD and non-SIMD WebAssembly binaries).
- The hand-landmark model.
- The web fonts (self-hosted; no font CDN at runtime).
- The web application itself.

A small reproducible Node script (`android/tools/fetch-assets.mjs`) fetches these artifacts and rewrites the web application's external references to local paths, so the offline bundle can be regenerated deterministically. At runtime everything is served from the in-app secure origin described in §5.1. The language model is the one large artifact provisioned out of band (side-loaded or downloaded once on first run) to keep the installable package small.

The result is verifiable: with the device in airplane mode, hand tracking, sentence composition, speech output and haptics all continue to function.

8. Permission and security model

- The app declares only the permissions it uses: `CAMERA` , `RECORD_AUDIO` , `VIBRATE` , and `INTERNET` (the last solely for the optional one-time model download).
- Runtime permissions are requested up front and, as a safety net, on demand at the moment the web layer needs them, so a previously skipped grant can still be recovered without leaving the app.
- The WebView loads only first-party content from the in-app origin; file-system and content-provider access are disabled, and there is no remote content surface.
- The single JavaScript bridge exposes a minimal, explicit method surface (model readiness, status, compose, provisioning) and carries no arbitrary command channel.

9. Performance characteristics

- **Vision.** Hand-landmark inference runs on the GPU delegate; measured per-frame latency is shown live in the UI and typically sits in the low tens of milliseconds, leaving headroom for a smooth frame loop.
- **Classification.** Pure arithmetic over 21 points; negligible cost.
- **Language.** The deterministic composer is effectively instantaneous. On-device LLM latency varies with hardware; the watchdog bounds the user-visible wait and the fallback covers slower devices.
- **Thermal/power.** Because there is no continuous network transfer and computation is GPU-delegated, sustained use stays within a practical thermal envelope — the core motivation for the on-device design.

10. Technology stack

Layer	Technology
App shell	Kotlin, Jetpack Compose, Android <code>WebView</code> , <code>WebViewAssetLoader</code>
Vision	MediaPipe Hand Landmarker (GPU delegate), WebAssembly
Language	Gemma-2B (4-bit) via LiteRT / MediaPipe LLM Inference; rule-based fallback
Speech	Platform text-to-speech and speech recognition
Sensors	Vibrator (haptics), accelerometer (shake detection)
Build	Gradle 8.10.2, Android Gradle Plugin 8.5.2, Kotlin 1.9.24, Compose BOM 2024.06.00
Targets	minSdk 26, compileSdk/targetSdk 34

11. Reliability and graceful degradation

The system is designed so that no single failure stops a live conversation:

- If the camera or vision runtime is unavailable, every gesture is also a tappable control, so the same pipeline can be driven by touch.
 - If the language model is absent or slow, the deterministic composer runs and a watchdog guarantees output.
 - If a permission is denied, the app remains usable in its touch-driven mode and the permission can be recovered on demand.
-

12. Limitations and future work

- **Vocabulary.** Recognition covers a curated lexicon, not unrestricted continuous signing.

Expanding the vocabulary and adding per-user personalisation is the largest area of future work.

- **Reverse transcription.** The embedded speech-recognition API is not available in every runtime environment; a native recognizer bridge with an offline language pack is planned.

- **Wearable relay.** A planned feature identifies a paired smartwatch over BLE (via the standard Device Information Service) and uses the on-device model to generate the appropriate notification payload, delivering the finalized sentence and its haptic cues to any standards-compliant wearable.
-

13. Project structure

```
index.html           Web application (UI + pipeline)
android/
  app/src/main/
    AndroidManifest.xml
    java/com/kraftshala/senselink/
      MainActivity.kt      Compose host, WebView, asset serving, permission broker
      LlmBridge.kt         On-device Gemma-2B bridge (load / compose / provision)
    assets/www/           Offline bundle (web app, MediaPipe, model runtime, fonts)
    res/                  Theme, colours, launcher icon
  tools/fetch-assets.mjs  Reproducible offline-asset bundler
docs/
  TECHNICAL_DOCUMENTATION.md  This document
  THEME.md                 Visual design system
releases/                Prebuilt debug APK
```

14. Build and run

```
cd android
node tools/fetch-assets.mjs      # bundle offline assets (fresh checkout / after editing the web app)
./gradlew :app:assembleDebug    # -> app/build/outputs/apk/debug/app-debug.apk
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Grant camera and microphone on first launch. To provision the on-device model, side-load the weights into the app's external files directory or configure a download URL (see the project README).